TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH

SỔ TAY THUYẾT TRÌNH

Báo cáo cuối kỳ môn Phân tích dữ liệu lớn

Đề tài 8

Phân tích dữ liệu IoT và bảo trì dự đoán

Học viên thực hiện:	Phan Nguyễn Mai Phương
Mã số học viên:	25002711
Ngành:	Khoa học máy tính

Mỗi slide một trang: hình slide, nhìn vào đâu, đọc gì, khi nào chuyển.

Tháng 10 năm 2026

Contents

Cách dùng sổ tay **2**

Trước buổi báo cáo **3**

Phần nói theo từng slide **4**

Slide 01 – Bìa . . . 5
Slide 02 – Nội dung trình bày . . . 6
Slide 03 – Bài toán . . . 7
Slide 04 – Dữ liệu NASA C-MAPSS . . . 8
Slide 05 – Nhãn RUL và ranh giới test . . . 9
Slide 06 – Pipeline Bronze – Silver – Gold . . . 10
Slide 07 – Pandas, PySpark và Hadoop . . . 11
Slide 08 – Khám phá dữ liệu . . . 12
Slide 09 – Khử nhiễu bằng EWMA . . . 13
Slide 10 – Cửa sổ thời gian 30 cycle . . . 14
Slide 11 – Xu hướng suy giảm . . . 15
Slide 12 – Phát hiện dấu hiệu bất thường . . . 16
Slide 13 – Dự đoán RUL . . . 17
Slide 14 – Phân tích sai số . . . 18
Slide 15 – Hai mức cảnh báo sớm . . . 19
Slide 16 – Báo cáo dự đoán thời điểm cần bảo trì . . . 20
Slide 17 – Thử nghiệm mở rộng . . . 21
Slide 18 – Trả lời năm câu hỏi . . . 22
Slide 19 – Khó khăn, bài học và giới hạn . . . 23

Demo dashboard **24**

Slide 20 – Cảm ơn . . . 30

Câu hỏi phản biện **31**

Số liệu tra nhanh **33**

Cách dùng sổ tay

Mỗi trang ứng với một slide, theo đúng thứ tự trên máy chiếu.

- **Ô vàng ở góc phải** là thời lượng nên dành cho slide, kèm mốc giờ khi kết thúc slide. Nếu đồng hồ chạy quá mốc 1 phút, đọc gọn lại.
- **Khung vàng "Nhìn vào"** chỉ chỗ cần đưa mắt hoặc chỉ tay trên slide.
- **Phần "Đọc"** là lời nói nguyên văn, có ví dụ đời thường để người không học chuyên ngành cũng theo được. Số in đậm là số quan trọng, nói chậm và rõ ở các số này.
- **Khung xám "Nói thêm"** là chi tiết sâu hơn. Chỉ đọc khi còn dư thời gian, hoặc dùng để trả lời khi bị hỏi.
- **Dòng "Chuyển slide"** ở cuối trang là câu nối sang slide tiếp theo.

Demo dashboard nằm **sau slide 19**, trước slide cảm ơn, có ảnh chụp từng bước. Cuối sổ tay có bộ câu hỏi phản biện và một trang số liệu để tra nhanh.

Kế hoạch thời gian

Phần	Slide	Hết phần lúc
Mở đầu	1–2	0:55
Bài toán và dữ liệu	3–5	3:20
Pipeline dữ liệu	6–8	5:35
Khử nhiễu và cửa sổ thời gian	9–10	7:25
Xu hướng và bất thường	11–12	9:25
Dự đoán RUL, cảnh báo, lịch bảo trì	13–16	13:50
Mở rộng, kết luận, bài học	17–19	16:20
Demo dashboard	–	18:50
Cảm ơn	20	19:00

Mốc trên chưa tính các khung "Nói thêm". Nếu bị giới hạn 15 phút: đọc lướt slide 8, 11, 14, 17 (mỗi slide khoảng 20 giây) và rút demo còn bước 1 (Lịch bảo trì). Không bỏ slide 6–7 (pipeline), 9 (khử nhiễu), 12 (bất thường), 15–16 (cảnh báo, lịch bảo trì) vì đó là các phần chấm điểm chính.

Trước buổi báo cáo

Ngày hôm trước

- Mở thử `slides/cmapss_fd001_presentation.pptx` trên máy sẽ dùng để chiếu, kiểm tra font Arial hiển thị đúng tiếng Việt.
- Chép thêm bản PDF `slides/cmapss_fd001_presentation.pdf` vào USB, phòng khi máy trình chiếu không có PowerPoint.
- Chạy thử dashboard một lần theo lệnh bên dưới, đi qua đủ 5 bước demo.
- Tập riêng các câu ví dụ (đồng hồ xăng, nhiệt độ cơ thể, đèn giao thông, hãng taxi): nói tự nhiên bằng giọng của mình, không cần thuộc từng chữ.
- Đọc to sổ tay một lượt có bấm giờ.

15 phút trước khi lên

1. Mở terminal ở thư mục gốc project và chạy:
   ```
   uv run --extra dashboard streamlit run app.py
   ```
2. Đợi trình duyệt mở `http://localhost:8501`, để sẵn ở trang **Tổng quan**. Nhấn `Ctrl` và phím `+` hai lần để chữ to hơn.
3. Mở PowerPoint ở chế độ Presenter View (Slide Show → Use Presenter View), để mình thấy speaker notes còn khán giả chỉ thấy slide.
4. Để sẵn hai cửa sổ: PowerPoint và trình duyệt. Chuyển qua lại bằng `Alt+Tab`.

Nếu có sự cố

- **Dashboard không mở được:** bỏ qua demo, nói "Em xin trình bày kết quả demo qua slide" rồi quay lại slide 12, 15, 16.
- **Máy không có PowerPoint:** chiếu bản PDF; speaker notes không còn, nên đọc theo sổ tay này.
- **Quên số liệu:** lật trang "Số liệu tra nhanh" ở cuối sổ tay.

Phần nói theo từng slide

Slide 01/20 Bìa

30 giây hết slide lúc 0:30



Nhìn vào

Đứng thẳng, nhìn về phía giảng viên và cả lớp. Khi nói bốn con số, đưa tay lần lượt qua bốn ô trắng từ trái sang phải.

Đọc

Kính chào thầy/cô và các bạn. Em là Phan Nguyễn Mai Phương, mã số học viên 25002711.

Mọi người thử hình dung: động cơ máy bay giống như một người làm việc rất chăm, nhưng không biết kêu mệt. Nếu chờ đến lúc nó hỏng mới sửa thì quá muộn; còn cứ vài tháng tháo ra kiểm tra một lần thì rất tốn kém. **Đề tài 8** của em đi tìm cách ở giữa: đọc dữ liệu cảm biến để biết khi nào động cơ **bắt đầu yếu đi**, và đoán xem **khi nào cần mang đi bảo trì**.

Bốn con số ở dưới là kết quả chính: **100** động cơ để học; lọc nhiễu bỏ được khoảng **86%** dao động lộn xộn; dấu hiệu bất thường được báo trước khi hỏng trung vị **121 cycle**; và dự đoán thời gian còn lại lệch khoảng **15 cycle**.

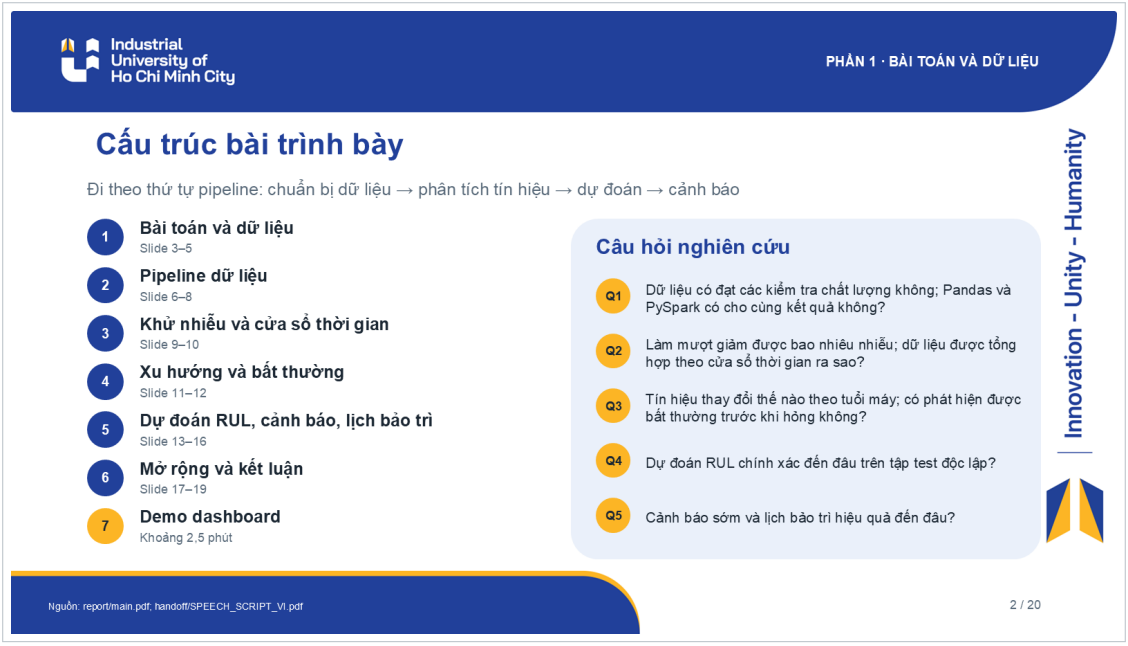
Nói thêm nếu còn thời gian hoặc được hỏi

Mỗi cycle là một chu kỳ vận hành của động cơ, có thể hình dung như một chuyến bay. Dữ liệu không quy đổi ra giờ hay ngày.

→ Chuyển slide: "Em xin đi qua nội dung trình bày."

Slide 02/20 Nội dung trình bày

25 giây hết slide lúc 0:55



Nhìn vào

Bên trái: lướt tay qua 7 vòng tròn từ trên xuống. Bên phải: chỉ vào khung "Câu hỏi nghiên cứu".

Đọc

Bài của em đi giống một dây chuyền trong nhà máy. Đầu dây chuyền là dữ liệu thô; qua các trạm kiểm tra, cất trữ, lọc nhiễu, tìm dấu hiệu bất thường; và cuối dây chuyền là sản phẩm: **lịch bảo trì cho từng động cơ**. Cuối bài em có khoảng hai phút rưỡi demo trực tiếp.

Năm câu hỏi bên phải là năm trạm của dây chuyền đó; ở slide 18 em sẽ trả lời lại từng câu.

→ Chuyển slide: "Trước hết, bài toán là gì."

Slide 03/20 Bài toán

45 giây hết slide lúc 1:40



Nhìn vào

Bên trái: chỉ thanh ngang, đoạn vàng bắt đầu ở $t = 170$, điểm đỏ cycle 200, rồi công thức xanh. Bên phải: chỉ cột "Slide" của bảng.

Đọc

Khái niệm trung tâm là **RUL**, tuổi thọ hữu ích còn lại. Nó giống đồng hồ xăng trên ô tô: đồng hồ không nói "xe đã chạy bao xa", mà nói "còn chạy được bao xa nữa". RUL cũng vậy: còn bao nhiêu cycle nữa thì động cơ hỏng.

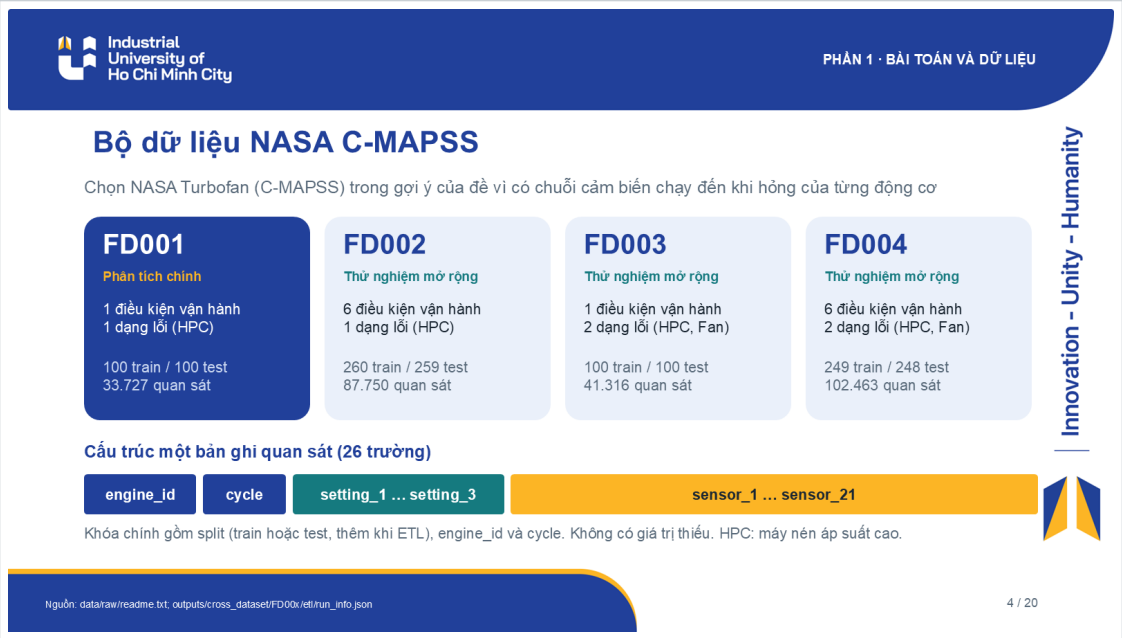
Ví dụ trên slide: động cơ hỏng ở cycle **200**. Tại cycle **170**, nó còn **30** cycle, tức RUL bằng 30. Công thức chỉ đơn giản là lấy cycle cuối trừ cycle hiện tại.

Bảng bên phải là các yêu cầu của đề tài, cột cuối ghi slide nào chứng minh yêu cầu đó, để thầy/cô tiện đối chiếu: xử lý chuỗi thời gian ở slide 6, 9, 10; lọc nhiễu ở slide 9; cửa sổ thời gian ở slide 10; bất thường ở slide 12; và lịch bảo trì ở slide 16.

→ Chuyển slide: "Vậy dữ liệu từ đâu ra?"

Slide 04/20 Dữ liệu NASA C-MAPSS

50 giây hết slide lúc 2:30



Nhìn vào

Chỉ dòng tóm tắt dưới tiêu đề (lý do chọn dữ liệu), rồi thẻ xanh đậm **FD001**, lướt ba thẻ nhạt bên cạnh, cuối cùng chỉ dải màu "26 trường".

Đọc

Muốn biết động cơ hỏng ra sao thì phải có động cơ bị hỏng. Nhưng không ai đem 100 động cơ máy bay thật ra chạy đến khi hỏng chỉ để lấy dữ liệu, vì quá đắt và quá nguy hiểm. Nên NASA làm một bộ **dữ liệu mô phỏng** tên là C-MAPSS, em tải trực tiếp từ cổng dữ liệu mở của NASA. Mỗi động cơ có cả chuỗi số đo theo từng cycle cho tới khi hỏng, và phần test có đáp án để chấm.

Bộ này có bốn phần. Em chọn **FD001** vì nó giống một phòng thí nghiệm sạch: chỉ một điều kiện bay và một kiểu hỏng, nên khi số đo thay đổi, mình biết đó là do động cơ yếu đi chứ không phải do thời tiết hay độ cao. Có **100** động cơ để học và **100** động cơ để kiểm tra.

Mỗi dòng dữ liệu là một cycle, gồm **21** cảm biến, giống như 21 chỉ số trong một tờ xét nghiệm máu. Dữ liệu không có ô nào bị thiếu.

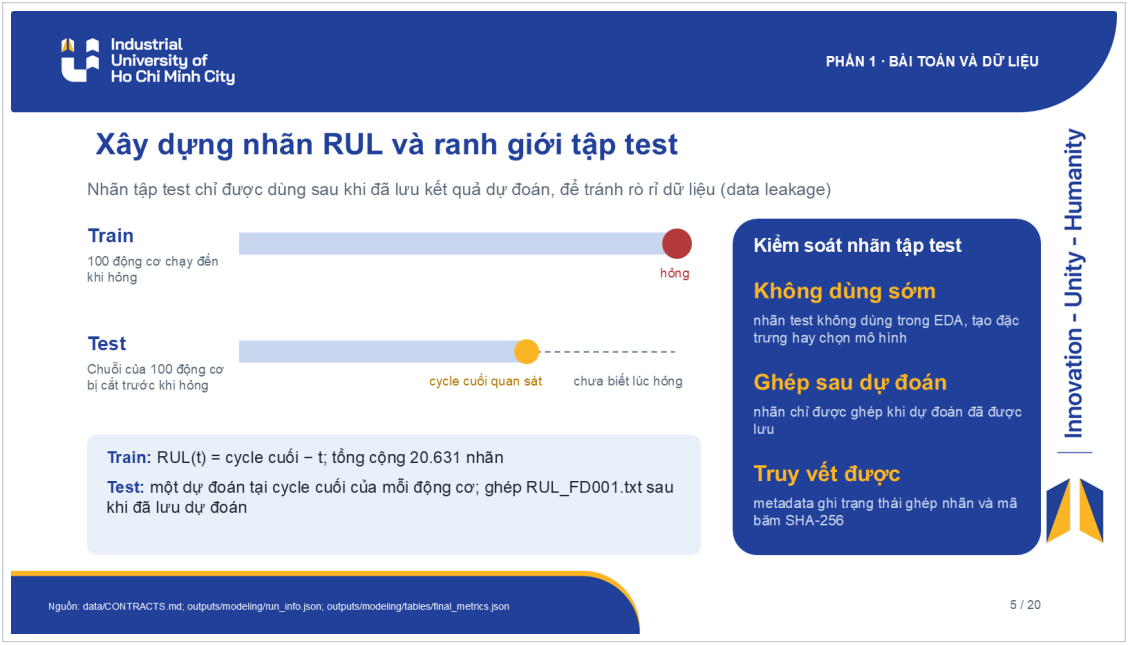
Nói thêm nếu còn thời gian hoặc được hỏi

Tổng cộng 33.727 dòng. Ba phần FD002–FD004 có thêm điều kiện bay hoặc kiểu hỏng khác, em chỉ dùng ở phần mở rộng (slide 17).

→ Chuyển slide: "Có dữ liệu rồi, em cần đáp án để mô hình học."

Slide 05/20 Nhãn RUL và ranh giới test

50 giây hết slide lúc 3:20



Nhìn vào

Chỉ thanh "Train" (chấm đỏ là hỏng), rồi thanh "Test" (dừng ở chấm vàng, sau đó là nét đứt). Cuối cùng chỉ khung xanh đậm bên phải.

Đọc

Dữ liệu có hai loại. Phần **train** giống hồ sơ bệnh án đã khép lại: mình biết bệnh nhân theo dõi từ đầu đến cuối, nên tính được ở mỗi thời điểm còn bao lâu. Phần **test** giống bệnh nhân đang nằm viện: mình chỉ thấy đến hôm nay, chưa biết kết cục, và phải đoán.

NASA có giữ đáp án cho phần test. Em coi đáp án này như **đề thi đã niêm phong**: không mở ra khi học, khi chọn mô hình hay khi vẽ biểu đồ. Chỉ khi đã nộp bài, tức đã ghi dự đoán ra file, em mới mở đáp án ra chấm. Máy còn ghi lại dấu vân tay của file đáp án để chứng minh không ai đụng vào trước đó.

Nói thêm nếu còn thời gian hoặc được hỏi

Phần train có 20.631 nhãn. "Dấu vân tay" ở đây là mã hash SHA-256 của file nhãn.

→ *Chuyển slide: "Tiếp theo là dây chuyền xử lý dữ liệu."*

Slide 06/20 Pipeline Bronze – Silver – Gold

55 giây hết slide lúc 4:15



Nhìn vào

Chỉ cột bên trái từ trên xuống: bước 1, 2, 3, rồi ô nét đứt Hadoop. Sau đó chỉ hàng A từ trái sang phải, rồi hàng B, rồi ô xanh đậm "Lịch bảo trì" ở góc phải. Cuối cùng lướt tay qua năm ô vàng ở hàng dưới cùng (Thu thập → Lưu trữ).

Đọc

Ba lớp Đồng, Bạc, Vàng giống quy trình làm trang sức. Cột bên trái là ba bước chuẩn bị chung. **Bước 1** tải dữ liệu thẳng từ trang của NASA và kiểm tra mã băm để chắc không sai một byte. **Bước 2** là lớp Đồng: file text gốc, giữ nguyên, không bao giờ ghi đè. **Bước 3** là cửa kiểm hàng: đủ 26 cột chưa, đúng kiểu số chưa, có dòng nào trùng không, các cycle có liền mạch không, rồi tính RUL cho train. Ô nét đứt bên dưới là bản chạy lại trên Hadoop: MapReduce tính lại bảng 200 động cơ và khớp hoàn toàn, nhưng không đi vào mô hình.

Nhánh A dự đoán RUL. A1 lưu lớp Bạc, là dữ liệu đã kiểm tra dạng Parquet, và lớp Vàng, là các bảng tổng hợp. A2 tạo 126 đặc trưng từ cửa sổ 30 cycle. A3 huấn luyện mô hình, chia 5 phần theo động cơ. A4 lưu 100 dự đoán test trước, rồi A5 mới mở đáp án để chấm điểm.

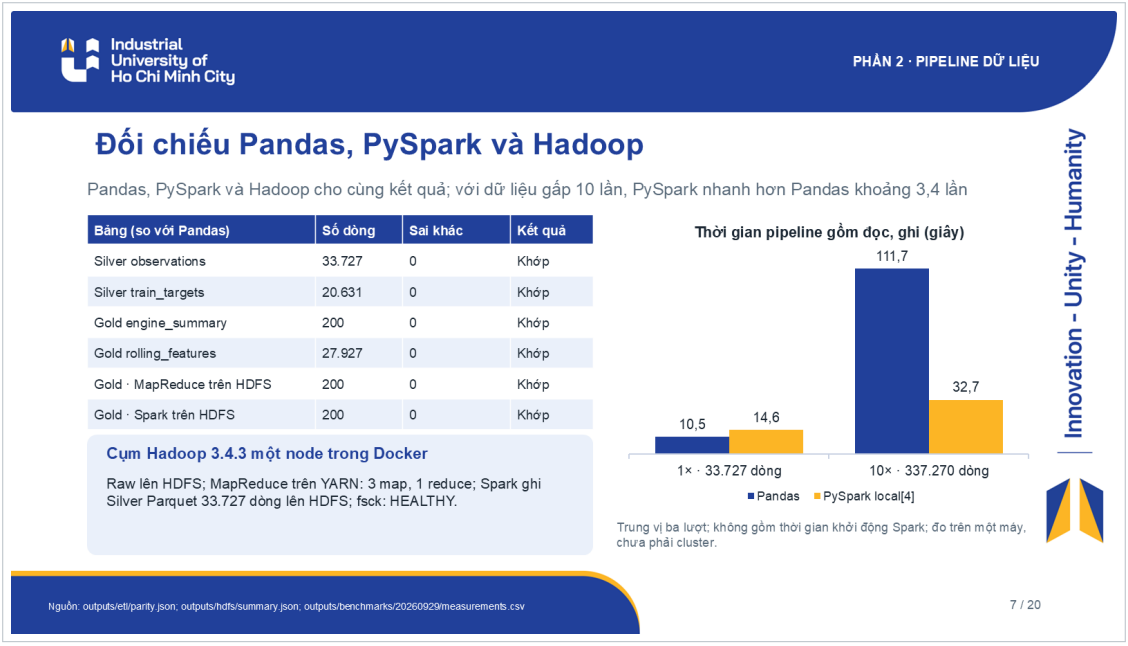
Nhánh B nghe ngóng sức khỏe động cơ và chỉ học trên train: khám phá dữ liệu, làm mượt tín hiệu, gộp 21 sensor thành một chỉ số sức khỏe, rồi tìm bất thường. Ô cuối ghép hai nhánh: động cơ có RUL dự đoán từ 30 trở xuống thì cần bảo trì, động cơ có chỉ số bất thường thì theo dõi thêm.

Năm ô vàng dưới cùng là năm bước của quy trình dữ liệu: **thu thập** ở bước 1, **làm sạch** ở bước 3 và B2, **chuẩn hóa** kiểu dữ liệu và thang đo ở slide 10, **tích hợp** là gộp train với test, và **lưu trữ** bằng Parquet, SQLite, HDFS. Pandas và PySpark cho cùng kết quả ở A1 và A2; slide tiếp theo nói rõ phần này.

→ Chuyển slide: "Em làm dây chuyền này bằng hai công cụ khác nhau."

Slide 07/20 Pandas, PySpark và Hadoop

55 giây hết slide lúc 5:10



Nhìn vào

Bên trái: chỉ cột "Sai khác" toàn số 0, nhấn vào hai dòng cuối có chữ HDFS, rồi ô xanh nhạt "Cụm Hadoop". Bên phải: chỉ cặp cột 10×, cột xanh 112 và cột vàng 33.

Đọc

Em đối chiếu Pandas/PySpark ở ETL và rolling features; MapReduce/Spark trên HDFS ở bảng tổng hợp động cơ. **Pandas** giống một đầu bếp giỏi làm một mình. **PySpark** giống một bếp có bốn đầu bếp chia nhau làm. Còn **Hadoop** là cả một nhà hàng có kho riêng: dữ liệu cất trong kho HDFS, và MapReduce chia việc thành hai công đoạn, sơ chế rồi gom lại. Em dựng nhà hàng đó trong Docker và cho nó tính lại bảng tổng hợp động cơ. Hai dòng cuối của bảng: cả MapReduce lẫn Spark đọc từ HDFS đều ra **200 trên 200** động cơ, sai khác **0**.

Biểu đồ bên phải: khi khách đông gấp **10 lần**, một mình mất khoảng **112 giây**, bếp bốn người chỉ mất khoảng **33 giây**, nhanh hơn khoảng **3,4 lần**. Đó là lý do khi dữ liệu lớn lên thì cần Spark.

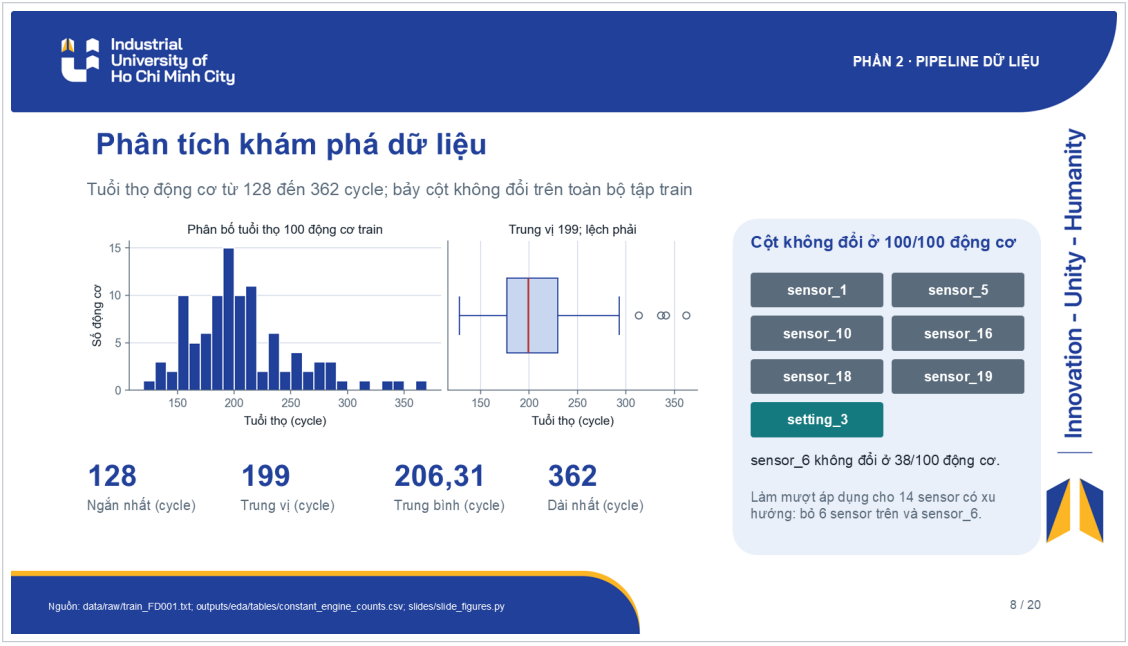
Nói thêm nếu còn thời gian hoặc được hỏi

Cụm Hadoop chỉ có một máy nên dùng để chứng minh tính đúng, không để đo tốc độ. Job MapReduce chạy trên YARN với 3 map và 1 reduce; lệnh kiểm tra `fsck` báo HDFS ở trạng thái HEALTHY. Rolling features cũng khớp toàn bộ 27.927 dòng giữa Pandas và Spark.

→ Chuyển slide: "Trước khi phân tích sâu, em xem qua dữ liệu trông thế nào."

Slide 08/20 Khám phá dữ liệu

25 giây hết slide lúc 5:35



Nhìn vào

Chỉ histogram bên trái và bốn con số bên dưới. Bên phải: chỉ các ô xám tên sensor.

Đọc

Động cơ sống ngắn nhất **128** cycle, dài nhất **362** cycle; một nửa số động cơ sống quá **199** cycle. Có **bảy cột** không bao giờ đổi, là các ô xám bên phải. Chúng giống cái nhiệt kế bị kẹt kim: đọc mãi một số nên không cho biết gì. Vì vậy em chỉ lọc nhiễu cho **14** cảm biến còn lại.

→ Chuyển slide: "Bây giờ đến phần lọc nhiễu."

Slide 09/20 Khử nhiễu bằng EWMA

1 phút 5 giây hết slide lúc 6:40



Nhìn vào

Chỉ đường **xám** lởm chởm, rồi đường **xanh** mượt ở hình sensor_11. Sau đó chỉ dòng sensor_4 trong bảng, rồi dòng "Công thức" phía dưới.

Đọc

Đường xám là số đo thật của động cơ 1: lên xuống lởm chởm từng cycle, giống nghe ai nói trong quán cà phê ồn ào. Mình không cần nghe rõ từng tiếng, chỉ cần nắm được ý chính.

Em dùng **EWMA**, trung bình trượt có trọng số mũ. Cách tính giống cách mình đánh giá phong độ một cầu thủ: trận gần nhất tính nhiều, các trận xa hơn tính ít dần. Cụ thể, mỗi cycle em lấy **18%** số đo mới cộng **82%** giá trị đã làm mượt trước đó. Kết quả là đường xanh: bỏ đi phần lởm chởm nhưng vẫn giữ đúng xu hướng đi lên ở cuối đời.

Điều quan trọng: EWMA **chỉ nhìn về quá khứ**, không lén nhìn tương lai. Bảng bên phải đo độ lởm chởm: ví dụ cảm biến 4 giảm từ **5,70** xuống **0,78**. Trên 14 cảm biến, độ dao động giảm khoảng **84 đến 86%**.

Nói thêm nếu còn thời gian hoặc được hỏi

Span 10 được chọn trước, không chỉnh theo kết quả. Giới hạn thẳng thắn: dữ liệu không có tín hiệu sạch để so, nên em đo được mức giảm dao động, chưa chứng minh được đã loại đúng nhiễu đo.

→ Chuyển slide: "Lọc xong, em gom dữ liệu theo từng khoảng thời gian."

Slide 10/20 Cửa sổ thời gian 30 cycle

45 giây hết slide lúc 7:25



Nhìn vào

Chỉ dải ô vuông trên cùng: ô vàng là $t = 170$, ô xanh đậm là cửa sổ, ô xám là tương lai không dùng. Lướt 6 ô thống kê. Bên phải: khung "Chuẩn hóa".

Đọc

Bác sĩ giỏi không nhìn một lần đo huyết áp rồi kết luận, mà xem sổ theo dõi 30 ngày gần nhất. Em làm đúng như vậy: tại cycle **170**, em nhìn lại **30 cycle** gần nhất, từ 141 đến 170, và không nhìn trước cycle 171.

Từ 30 cycle đó, mỗi cảm biến cho sáu con số: giá trị mới nhất, trung bình, độ dao động, thấp nhất, cao nhất, và xu hướng đang lên hay xuống. **21** cảm biến nhân **6** được **126** đặc trưng, giống một bản tóm tắt sức khỏe 126 dòng cho mỗi cycle.

Nói thêm nếu còn thời gian hoặc được hỏi

Chuẩn hóa: ETL thống nhất kiểu dữ liệu và tên cột; StandardScaler đưa các cảm biến về cùng thang đo trước khi tính chỉ số sức khỏe. Mô hình cây không cần bước này vì cây chỉ so lớn hay nhỏ.

→ Chuyển slide: "Vậy cảm biến thay đổi thế nào khi động cơ già đi?"

Slide 11/20 Xu hướng suy giảm

40 giây hết slide lúc 8:05



Nhìn vào

Chỉ hai hình bên trái, sensor_4 đi lên và sensor_7 đi xuống. Sau đó chỉ ô xanh **15/21**.

Đọc

Hai hình bên trái là cảm biến 4 và cảm biến 7 của ba động cơ: một cái đi lên, một cái đi xuống khi gần hỏng. Để chắc đây không phải do may rủi, em so 30 cycle cuối đời với 30 cycle đầu đời của **chính động cơ đó**, giống so ảnh lúc mới mua với ảnh lúc sắp hỏng của cùng một chiếc máy, rồi làm kiểm định thống kê.

Kết quả: **15 trên 21** cảm biến thay đổi rõ ràng, và có **12** cảm biến đổi cùng một hướng ở **cả 100** động cơ. Tức là động cơ thật sự có dấu hiệu già đi mà mình đo được.

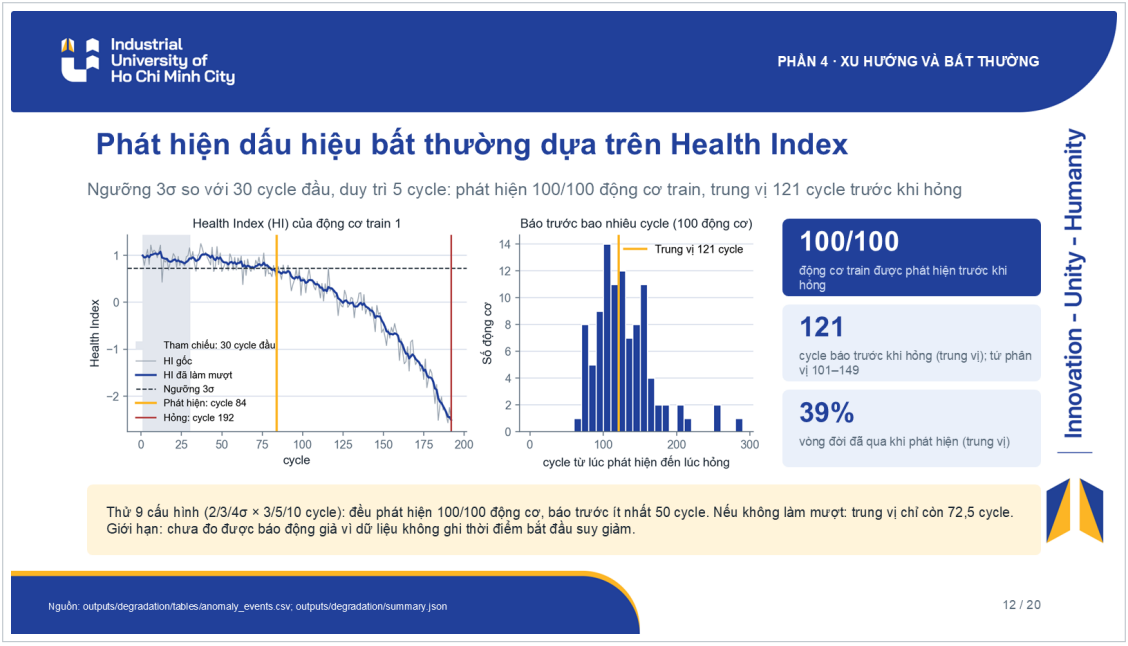
Nói thêm nếu còn thời gian hoặc được hỏi

Kiểm định Wilcoxon, khoảng tin cậy bootstrap 10.000 lần, hiệu chỉnh Benjamini–Hochberg cho 21 cảm biến. Sáu cảm biến hằng thì không kiểm định được.

→ Chuyển slide: "Từ đó, em xây cách phát hiện động cơ bắt đầu bất thường."

Slide 12/20 Phát hiện dấu hiệu bất thường

1 phút 20 giây hết slide lúc 9:25



Nhìn vào

Hình trái: **vùng xám** → **đường đứt** → **vạch cam** (cycle 84) → **vạch đen** (cycle 192). Hình phải: histogram. Sau đó: ba ô bên phải. Cuối cùng: **dải vàng** phía dưới (độ nhạy).

Đọc

Theo dõi 21 cảm biến cùng lúc thì rối mắt, nên em gộp chúng thành **một điểm sức khỏe**, gọi là Health Index, giống như gộp cả tờ xét nghiệm thành một câu "khỏe" hay "yếu". Điểm này giảm dần khi động cơ già đi.

Vậy khi nào gọi là bất thường? Em dùng cách của nhiệt độ cơ thể. Mỗi người có mức bình thường riêng; chỉ khi lệch quá xa **mức của chính mình** mới gọi là sốt. Ở đây, vùng xám là 30 cycle đầu đời, lúc động cơ còn khỏe. Đường đứt là mức lệch quá **3 lần** độ dao động bình thường. Và để tránh báo động giả kiểu nhiệt kế đo nhầm một lần, em chỉ báo khi điểm sức khỏe nằm dưới ngưỡng **5 cycle liền**.

Động cơ 1 bị phát hiện ở cycle **84**, vạch cam, trong khi nó hỏng ở cycle **192**: báo trước **108 cycle**. Trên cả 100 động cơ train, **không sót cái nào**, và một nửa số động cơ được báo trước hơn **121 cycle**. Đây là **đèn vàng**: động cơ bắt đầu yếu, nên để ý.

Dải vàng phía dưới trả lời câu hỏi "con số 3 và 5 có phải chọn cho đẹp không". Em đã thử đủ **9 cách đặt**, từ khắt khe đến dễ dãi; cách nào cũng bắt đủ 100 động cơ, và động cơ báo muộn nhất vẫn còn ít nhất **50 cycle** mới hỏng.

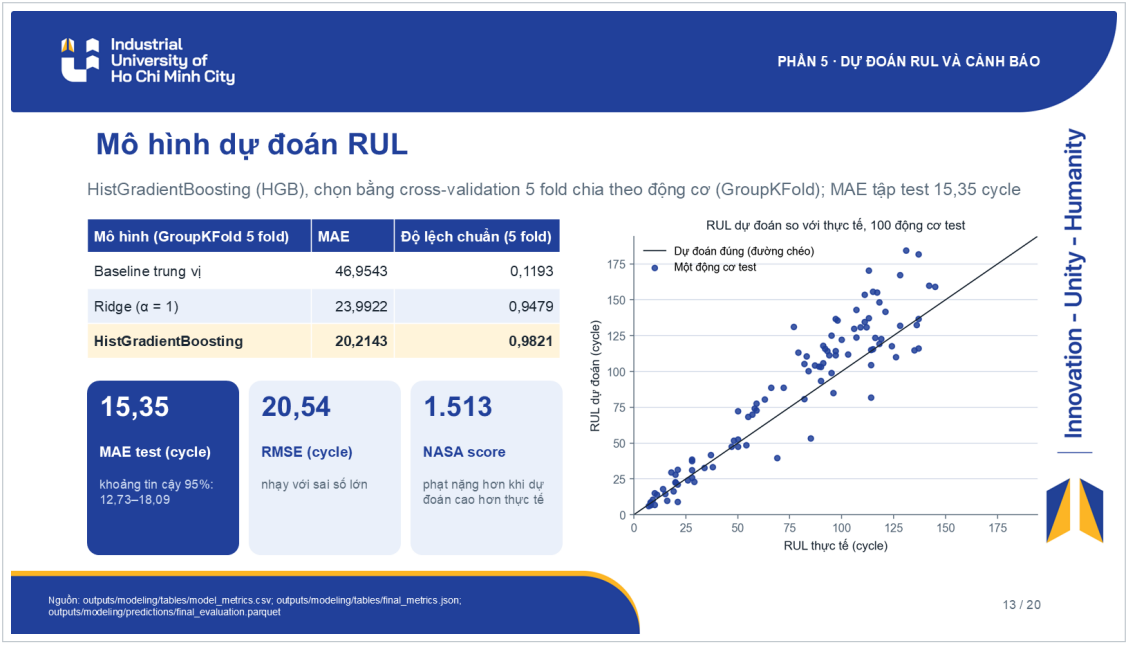
Nói thêm nếu còn thời gian hoặc được hỏi

Nếu không lọc nhiễu, con số chỉ còn 72,5 cycle, vì nhiễu làm vùng bình thường rộng ra. Đặt ngưỡng dễ dãi (2 sigma, 3 cycle) thì báo sớm hơn nhưng 17 động cơ bị báo ngay sau 30 cycle đầu, nhiều khả năng là báo hơi vội. Giới hạn: mọi động cơ trong dữ liệu đều hỏng, nên chưa đo được trực tiếp tỷ lệ báo động giả.

→ Chuyển slide: "Đèn vàng cho biết động cơ yếu, nhưng chưa nói còn bao lâu. Đó là việc của mô hình."

Slide 13/20 Dự đoán RUL

1 phút 10 giây hết slide lúc 10:35



Nhìn vào

Chỉ dòng tô vàng **HistGradientBoosting**, rồi ô xanh đậm **15,35**. Bên phải: chỉ đường chéo và cụm điểm ở góc dưới trái.

Đọc

Trước khi khoe kết quả, em phải nói cách chấm. Hai cửa sổ liền nhau của cùng một động cơ giống nhau 29 trên 30 cycle. Nếu một cái nằm trong bài học, cái kia nằm trong bài thi, thì khác gì cho học sinh xem đề trước. Nên em chia theo **từng động cơ**: động cơ nào đã ở phần thi thì không xuất hiện trong phần học.

Em thử ba mô hình, và mô hình **HistGradientBoosting**, một tập hợp rất nhiều cây quyết định nhỏ, cho sai số thấp nhất. Trên 100 động cơ test, trung bình mô hình đoán lệch **15,35 cycle**. Sai số này không đều: khi động cơ còn dưới 30 cycle thì chỉ lệch khoảng 4 cycle, còn khi động cơ còn khỏe thì lệch khoảng 21 cycle.

Ở hình phải, điểm càng sát đường chéo là đoán càng đúng. Mọi người để ý góc dưới bên trái: khi động cơ sắp hỏng, các điểm bám rất sát đường chéo.

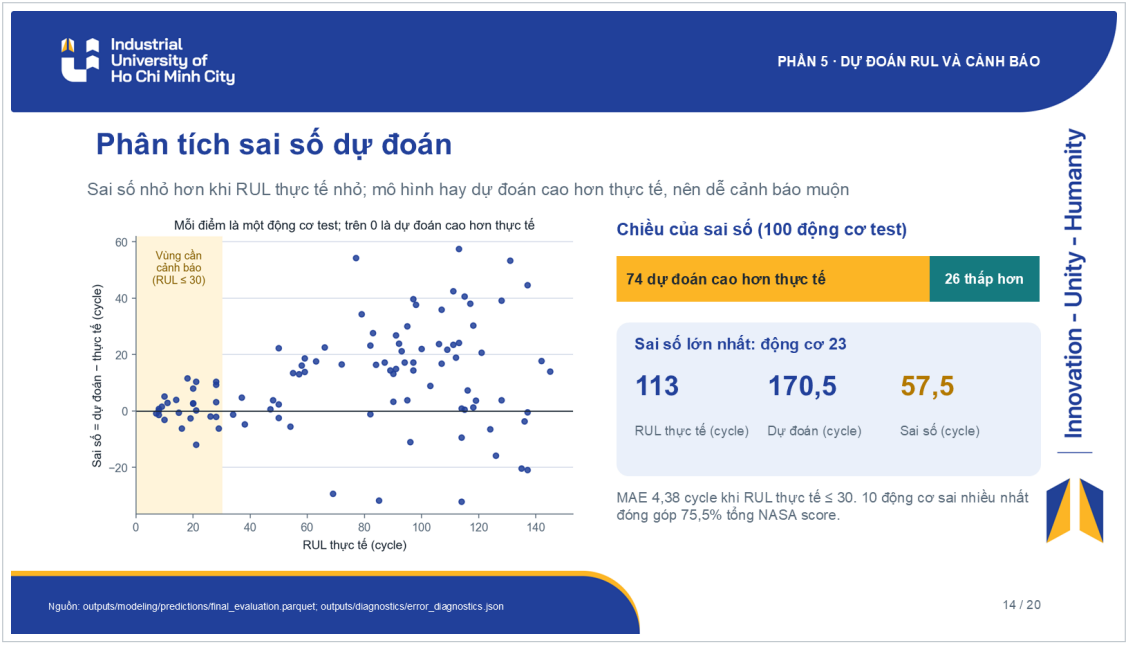
Nói thêm nếu còn thời gian hoặc được hỏi

Khoảng tin cậy 95% của MAE từ 12,73 đến 18,09 cycle; RMSE 20,54. Sai số cross-validation là 20,21 vì nó tính cả giai đoạn đầu đời rất khó đoán.

→ Chuyển slide: "Vậy mô hình hay sai ở đâu?"

Slide 14/20 Phân tích sai số

40 giây hết slide lúc 11:15



Nhìn vào

Hình trái: cụm điểm sát số 0 bên trái, rồi các điểm tản ra bên phải. Bên phải: thanh vàng "74 dự đoán cao hơn thực tế" và ô "Sai số lớn nhất: động cơ 23".

Đọc

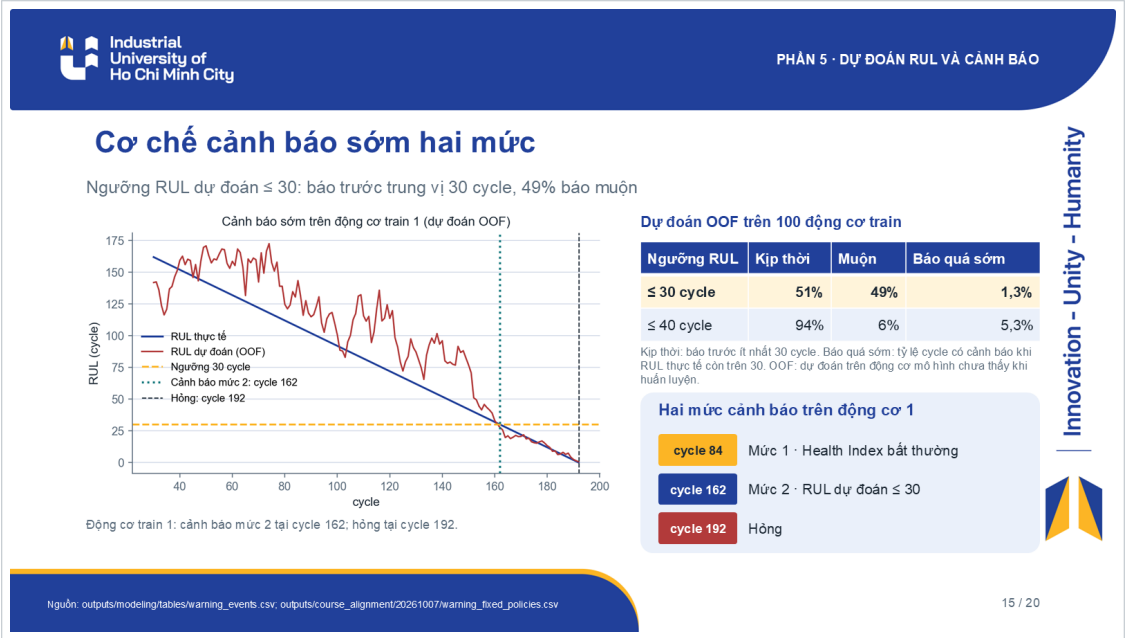
Giống dự báo thời tiết: dự báo ngày mai thì khá chính xác, dự báo tháng sau thì khó hơn nhiều. Khi động cơ sắp hỏng, mô hình chỉ lệch khoảng **4 cycle**; khi động cơ còn khỏe, nó lệch nhiều hơn vì lúc đó cảm biến gần như chưa đổi gì.

Mô hình cũng hơi lạc quan: **74 trên 100** lần nó nghĩ động cơ còn khỏe hơn thực tế, kiểu người hay nói "em vẫn ổn". Ví dụ động cơ 23 còn **113** cycle mà mô hình đoán **170**.

→ Chuyển slide: "Biết được điểm yếu đó, em thiết kế cảnh báo hai mức."

Slide 15/20 Hai mức cảnh báo sớm

1 phút 20 giây hết slide lúc 12:35



Nhìn vào

Hình trái: đường xanh (thật), đường đỏ (dự đoán), đường cam ngang (ngưỡng 30), vạch đứng cycle 162. Bảng phải: dòng chữ "Dự đoán OOF trên 100 động cơ train", rồi dòng "≤ 30 cycle". Cuối cùng: ba ô màu 84 / 162 / 192.

Đọc

Cảnh báo của em giống **đèn giao thông**. Đèn vàng là phần phát hiện bất thường ở slide 12: động cơ bắt đầu yếu. Đèn đỏ là khi mô hình đoán chỉ còn **30 cycle** hoặc ít hơn: cần đưa đi bảo trì.

Hình bên trái là động cơ 1: đường xanh là thời gian còn lại thật, đường đỏ là dự đoán, đường cam là ngưỡng 30. Đèn đỏ bật ở cycle **162**, đúng lúc thời gian thật chạm 30.

Bảng bên phải dùng 100 động cơ train, với dự đoán trên những động cơ mô hình chưa thấy khi học (gọi là OOF). Ở đó **không có động cơ nào bị bỏ sót**. Nhưng em phải nói thật: **gần một nửa** bật đèn đỏ hơi muộn, chủ yếu là các động cơ xuống sức nhanh. Nếu nâng ngưỡng lên 40 thì giống đặt báo thức sớm hơn: đúng giờ **94%** thay vì 51%, nhưng phải dậy sớm, tức báo sớm nhiều hơn khoảng bốn lần. Chọn mức nào phải tùy chi phí bảo trì, mà dữ liệu không có.

Ghép lại trên động cơ 1, ở ba ô màu: cycle 84 đèn vàng, cycle 162 đèn đỏ, cycle 192 hỏng.

→ Chuyển slide: "Ghép đèn vàng và đèn đỏ lại, em có lịch bảo trì."

Slide 16/20 Báo cáo dự đoán thời điểm cần bảo trì

1 phút 15 giây hết slide lúc 13:50



Nhìn vào

Hình trên: các cột cam bên trái và biểu đồ "Trạng thái hiện tại". Bảng dưới: dòng động cơ 90. Bảng phải: dòng "Cần bảo trì" (21, 0, 0), rồi khung xanh **21/25**.

Đọc

Đây là **sản phẩm đặc thù** mà đề tài yêu cầu. Hãy tưởng tượng mình là quản lý một hãng taxi có 100 chiếc xe: sáng nay cần biết xe nào vào xưởng hôm nay, xe nào tuần sau, xe nào còn chạy thoải mái. Bảng này làm đúng việc đó cho 100 động cơ.

Bảng dưới trích 5 dòng của lịch. Ví dụ động cơ 90: đang ở cycle **146**, mô hình đoán còn **31** cycle, nên lịch ghi **bảo trì ở cycle 147**. Kết quả cả đội: **21** động cơ đèn đỏ cần bảo trì ngay, **59** đèn vàng cần theo dõi, **20** đang khỏe; và **14** động cơ sẽ đến hạn trong 25 cycle tới.

Phần em thích nhất là bảng bên phải: sau khi lịch đã ghi xong, em mới mở đáp án ra chấm. Cả **21 động cơ** bị báo đỏ **đều đúng là cần bảo trì**, không báo oan chiếc nào. Bốn chiếc còn lại thật sự cần bảo trì thì lịch xếp trễ chỉ **1 đến 8 cycle**, và vẫn đang ở đèn vàng.

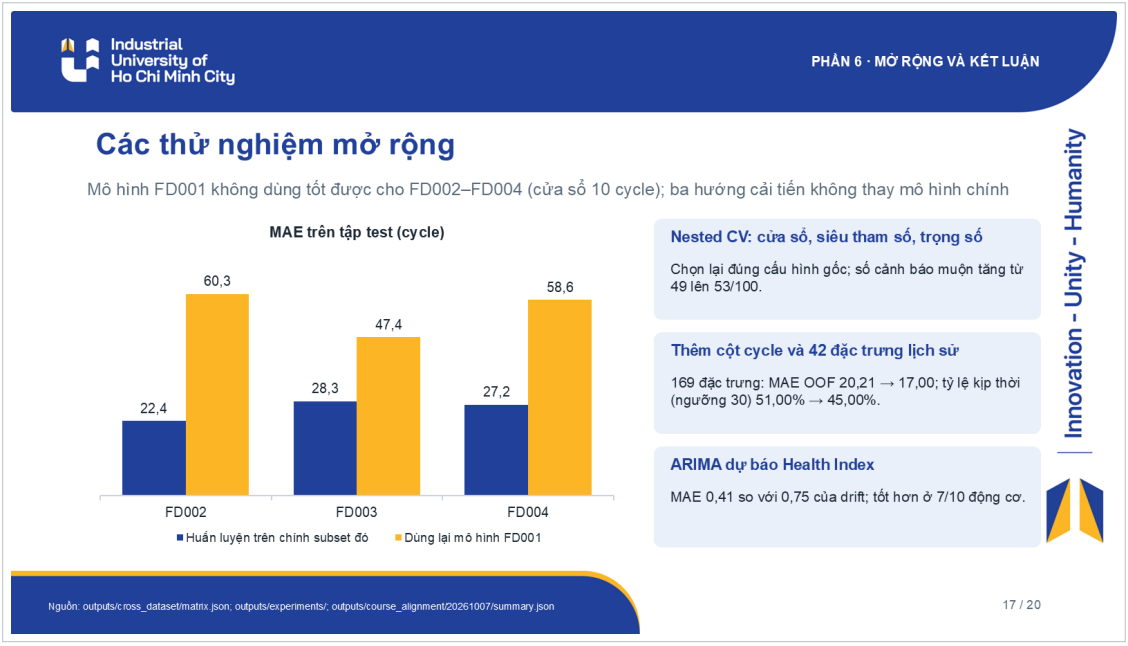
Nói thêm nếu còn thời gian hoặc được hỏi

Giới hạn: đèn vàng còn hơi rộng, 41 trên 59 động cơ đèn vàng thực ra còn hơn 60 cycle. Nên đèn vàng dùng để kiểm tra thường xuyên hơn, chưa dùng để lên lịch sửa. Ngoài ra, 12 động cơ còn xa mới hỏng có lịch muộn hơn lúc hỏng 4–27 cycle, vì mô hình đoán cao hơn thực tế ở vùng RUL lớn; lịch cần được tính lại khi có dữ liệu mới.

→ Chuyển slide: "Ngoài luồng chính, em còn thử thêm vài hướng."

Slide 17/20 Thử nghiệm mở rộng

50 giây hết slide lúc 14:40



Nhìn vào

Biểu đồ trái: so cột xanh với cột vàng ở từng subset. Bên phải: lướt ba khung xanh nhạt.

Đọc

Biểu đồ trái trả lời câu hỏi: mô hình học trên FD001 có dùng được cho động cơ bay ở điều kiện khác không? Câu trả lời là không: sai số tăng **1,7 đến 2,7 lần**. Giống người học lái xe ở đồng bằng rồi lên đèo: vẫn là lái xe, nhưng kỹ năng không dùng y nguyên được.

Bên phải là ba hướng em thử để cải thiện mô hình. Có hướng giảm được sai số nhưng lại làm đèn đỏ bật muộn hơn, nên em **không đổi mô hình chính**. Bài học: con số đẹp hơn chưa chắc là cảnh báo tốt hơn.

Nói thêm nếu còn thời gian hoặc được hỏi

Thêm cột cycle và 42 đặc trưng lịch sử: sai số giảm từ 20,21 xuống 17,00 nhưng tỷ lệ cảnh báo kịp thời giảm từ 51% xuống 45%. ARIMA dự báo điểm sức khỏe tốt hơn mốc so sánh nhưng chỉ ở 7 trên 10 động cơ.

→ Chuyển slide: "Em xin tổng kết lại."

Slide 18/20 Trả lời năm câu hỏi

50 giây hết slide lúc 15:30



Nhìn vào

Chỉ lần lượt 5 thẻ Q1 → Q5, rồi khung xanh đậm "Sản phẩm đặc thù".

Đọc

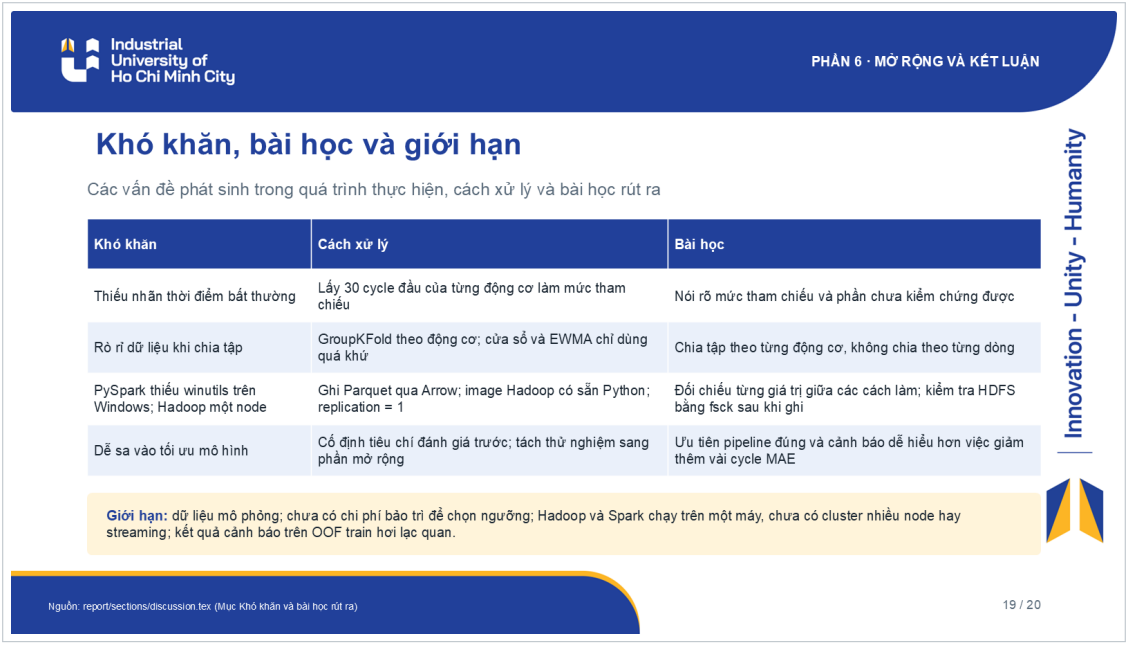
Năm câu hỏi, năm câu trả lời. Một: dữ liệu sạch, hai công cụ ra kết quả **giống hệt nhau**. Hai: lọc nhiễu bỏ được khoảng **86%** dao động. Ba: phát hiện bất thường ở **cả 100** động cơ train, trung vị **121 cycle** trước khi hỏng. Bốn: dự đoán lệch khoảng **15 cycle**, và rất chính xác khi gần hỏng. Năm: trên test, lịch bảo trì báo đỏ **21 chiếc, đúng cả 21**, bắt được 21 trên 25 chiếc cần bảo trì; trên train, cảnh báo không bỏ sót chiếc nào nhưng gần một nửa báo muộn.

Gói lại trong một câu: từ dữ liệu cảm biến thô, em làm ra được **lịch bảo trì cho từng động cơ**, kèm biểu đồ cảnh báo sớm.

→ Chuyển slide: "Trên đường đi, em cũng vấp vài lần."

Slide 19/20 Khó khăn, bài học và giới hạn

50 giây hết slide lúc 16:20



Nhìn vào

Đọc theo từng hàng của bảng: cột trái là khó khăn, cột phải là bài học. Cuối cùng chỉ dòng vàng "Giới hạn".

Đọc

Bốn lần vấp. Một: dữ liệu không ghi lúc nào động cơ bắt đầu yếu, nên em so mỗi động cơ với chính nó lúc khỏe. Hai: dữ liệu theo thời gian rất dễ lộ đề, nên phải chia theo động cơ chứ không chia theo dòng. Ba: hạ tầng. Spark trên Windows không chịu ghi file, còn Hadoop trong Docker thì lần đầu báo chạy xong mà kho vẫn thiếu bản sao; em học được là chạy xong chưa đủ, phải kiểm tra lại kho. Bốn, và đây là bài học lớn nhất: em từng mất khá nhiều thời gian chỉ để giảm vài cycle sai số, rồi nhận ra người vận hành cần **một cảnh báo dễ hiểu** hơn là một con số đẹp hơn.

Giới hạn: đây là dữ liệu mô phỏng, chưa có chi phí bảo trì, Hadoop và Spark mới chạy trên một máy, và kết quả cảnh báo trên train hơi lạc quan.

→ Chuyển slide: "Bây giờ em xin demo nhanh khoảng hai phút rưỡi." Bấm `Alt+Tab` sang trình duyệt.

Demo dashboard (2 phút 30 giây, sau slide 19)

Thứ tự 5 bước: Lịch bảo trì → Khử nhiễu → Xu hướng & bất thường → Cảnh báo sớm → Tổng quan. Các trang chọn ở thanh bên trái, mục "Điều hướng". Không mở terminal và không chạy lại huấn luyện trước lớp.

Bước 1 – Lịch bảo trì (50 giây, hết demo lúc 17:10)



Bấm

Thanh trái: bấm **Lịch bảo trì**. Chỉ vào bốn ô số trên cùng, rồi bảng.

Đọc

Đây là bảng của người quản lý đội động cơ mà em vừa kể. Trên cùng là bốn con số: **21** cần bảo trì ngay, **59** cần theo dõi, **20** đang khỏe, **14** sắp đến hạn. Bảng xếp theo mức ưu tiên: chiếc cần vào xưởng sớm nhất nằm trên cùng.



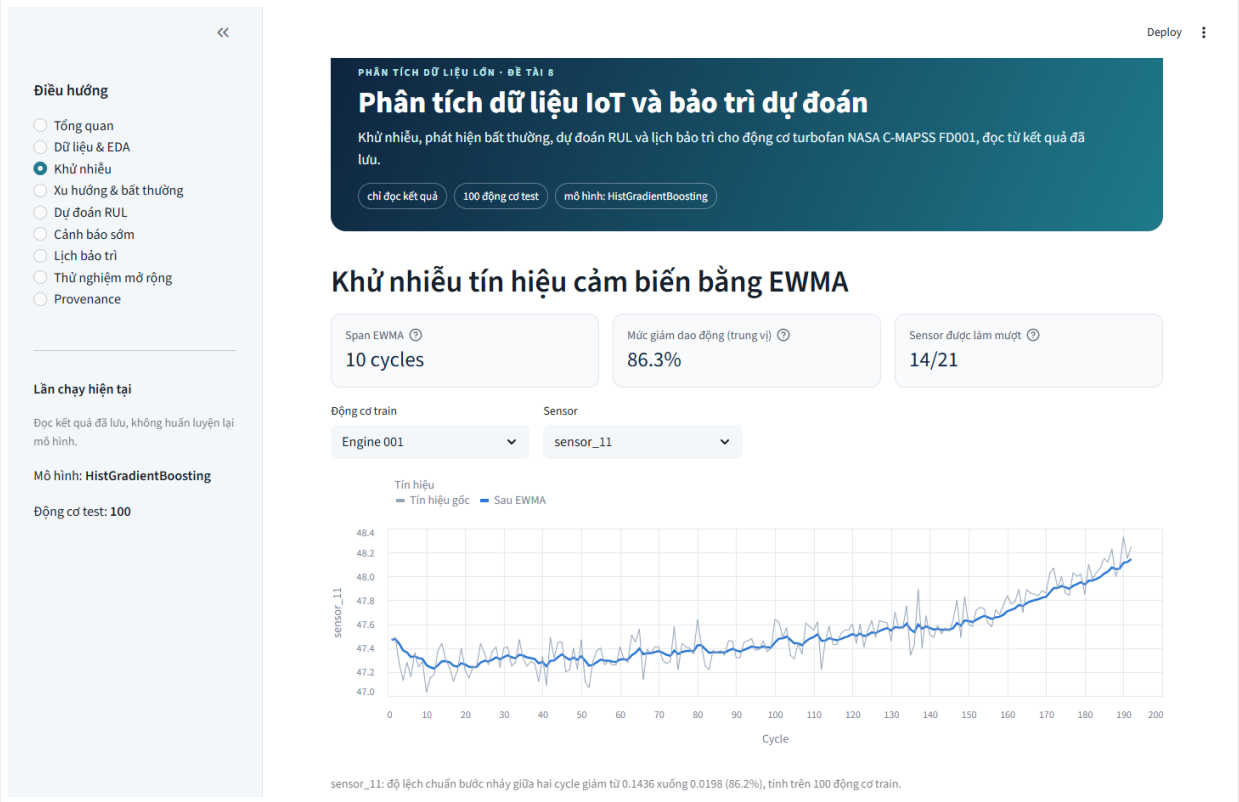
Bấm

Kéo xuống. Ô **"Xem chi tiết động cơ test"** bên phải: bấm vào, gõ `090`, nhấn Enter. Chỉ ô 31,2 và 147, rồi vạch cam trên biểu đồ. Kéo xuống bảng "Kiểm tra bằng nhãn test".

Đọc

Em chọn động cơ 90: mô hình đoán còn **31** cycle, nên lịch ghi bảo trì ở cycle **147**. Vạch cam cho thấy điểm sức khỏe của nó đã bật đèn vàng từ cycle **67**. Và bảng cuối trang là phần chấm bằng đáp án: báo đỏ **21 chiếc, đúng cả 21**.

Bước 2 – Khử nhiễu (30 giây, hết lúc 17:40)



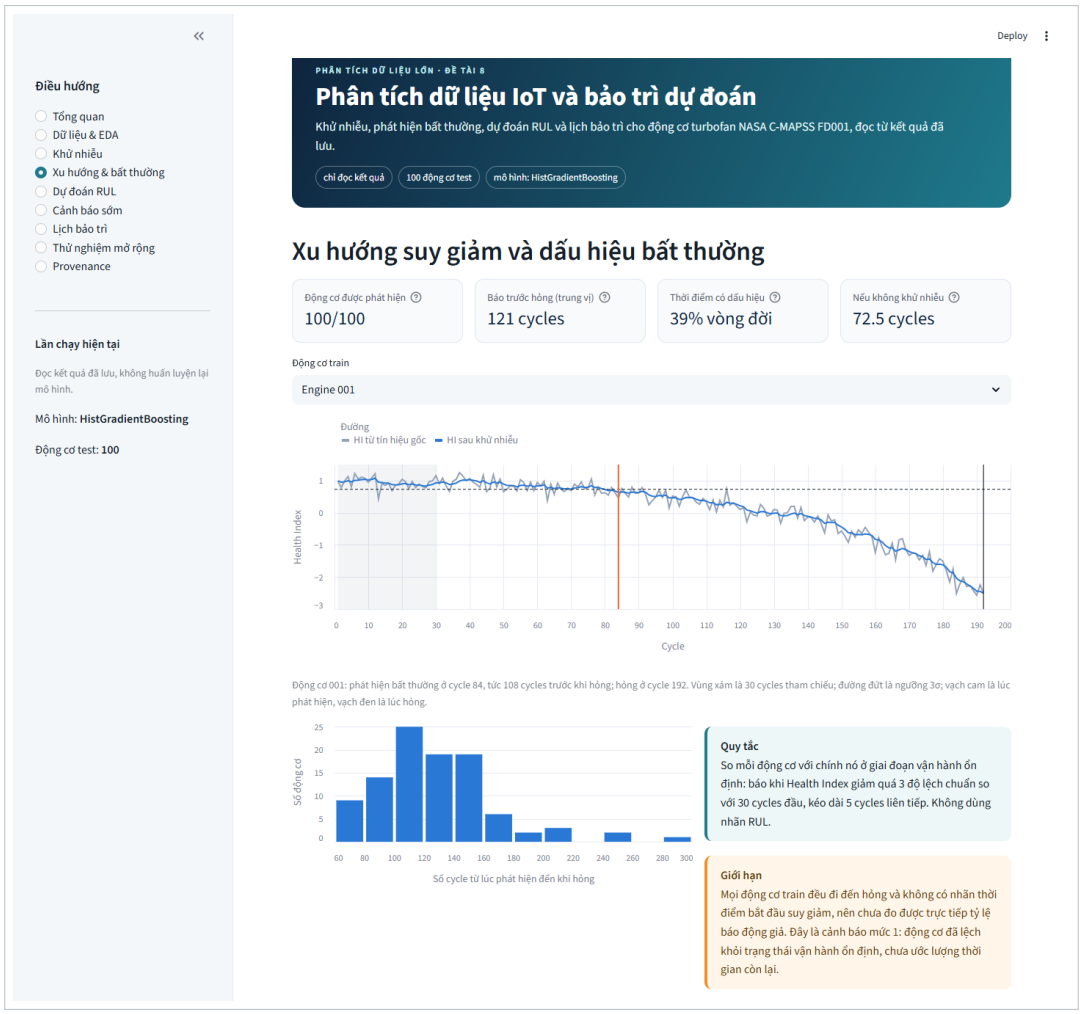
Bấm

Thanh trái: bấm **Khử nhiễu**. Giữ Engine 001 và sensor_11. Chỉ đường xám rồi đường xanh. Nếu được hỏi thêm, đổi ô Sensor sang `sensor_4`.

Đọc

Đây là quán cà phê ồn ào em nói lúc nãy: đường xám là tiếng ồn, đường xanh là ý chính sau khi lọc. Có thể đổi sang bất kỳ động cơ hay cảm biến nào, hình dạng đều tương tự.

Bước 3 – Xu hướng và bất thường (35 giây, hết lúc 18:15)



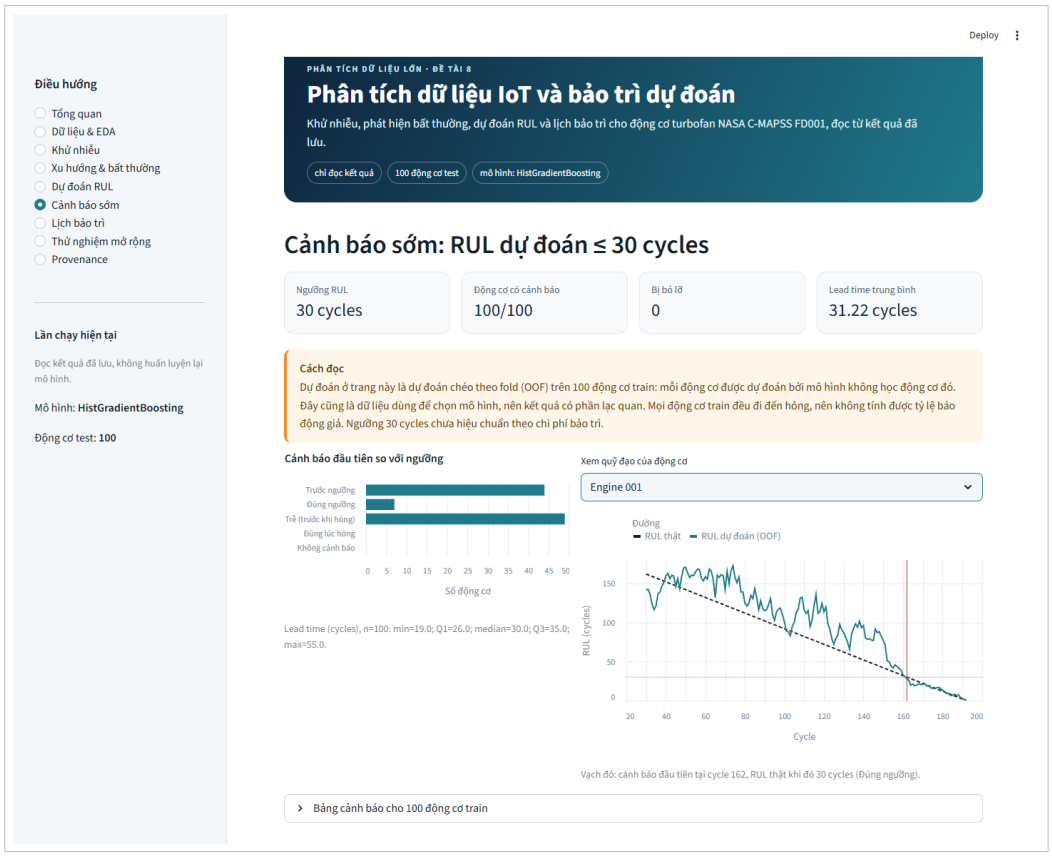
Bấm

Thanh trái: bấm **Xu hướng & bất thường**. Giữ Engine 001. Chỉ vùng xám → đường đứt → vạch cam → vạch đen, rồi histogram bên dưới.

Đọc

Vùng xám là lúc động cơ còn khỏe, đường đứt là ngưỡng sốt. Điểm sức khỏe rớt qua ngưỡng và nằm dưới đó 5 cycle liền ở cycle **84**, đèn vàng bật, còn **108** cycle nữa mới hỏng. Biểu đồ dưới cho thấy cả 100 động cơ đều được báo trước như vậy.

Bước 4 – Cảnh báo sớm (25 giây, hết lúc 18:40)



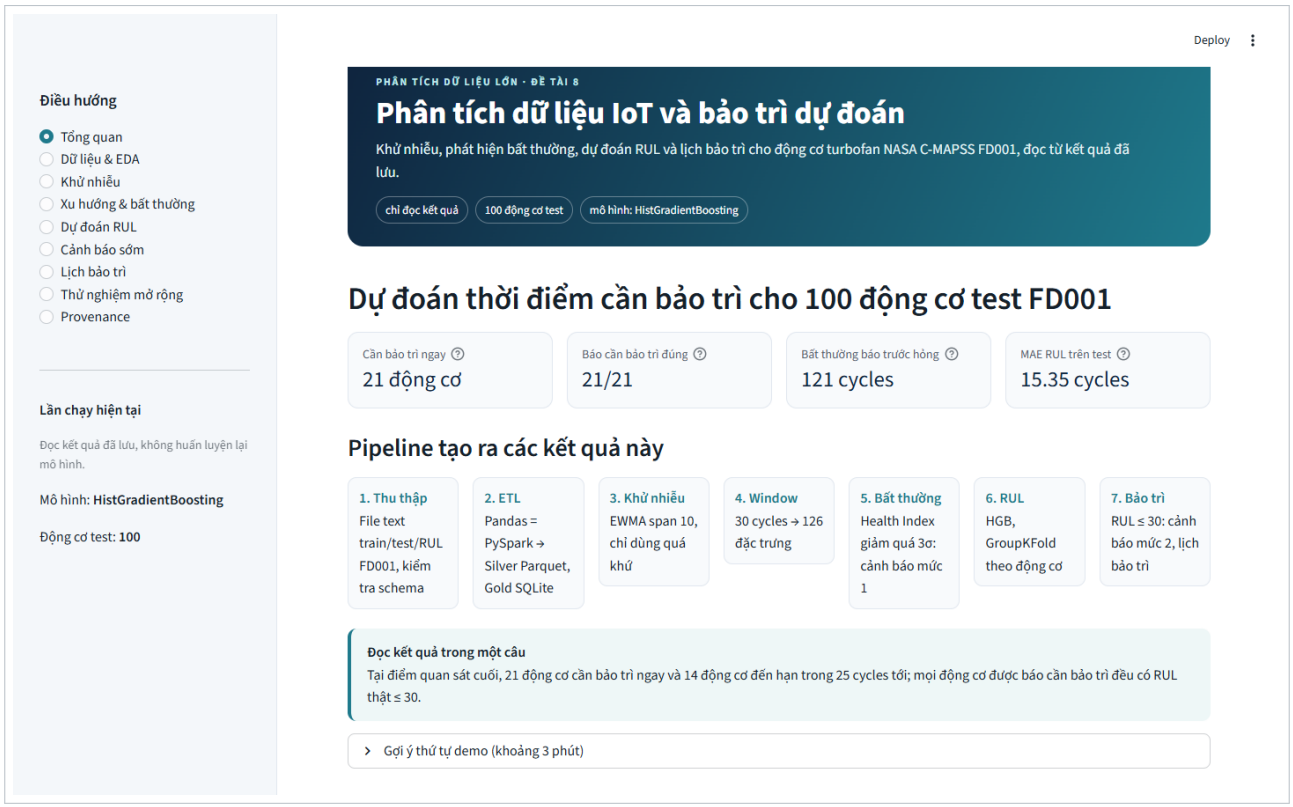
Bấm

Thanh trái: bấm **Cảnh báo sớm**. Ô "Xem quỹ đạo của động cơ": chọn `Engine 001`. Chỉ vạch đứng ở cycle 162.

Đọc

Cùng động cơ 1, đèn đỏ bật ở cycle **162**. Vậy là: vàng ở 84, đỏ ở 162, hỏng ở 192.

Bước 5 – Kết demo (10 giây, hết lúc 18:50)



Bấm

Thanh trái: bấm **Tổng quan**. Sau đó `Alt+Tab` về PowerPoint, sang slide 20.

Đọc

Mọi con số trên dashboard đều đọc từ cùng kết quả với báo cáo và slide, không có số nào làm tay. Em xin quay lại slide cuối.

Slide 20/20 Cảm ơn

10 giây hết slide lúc 19:00



Nhìn vào
Nhìn về phía giảng viên, mỉm cười, dừng lại chờ câu hỏi.

Đọc

Em xin cảm ơn thầy/cô và các bạn đã lắng nghe. Em rất mong nhận được câu hỏi và góp ý.

Câu hỏi phản biện

Trả lời ngắn trước, chỉ thêm chi tiết khi được hỏi tiếp. Nếu cần hình minh họa, quay lại slide ghi trong ngoặc.

Hỏi: Vì sao chọn EWMA để khử nhiễu? (slide 9)

Trả lời: EWMA chỉ dùng quá khứ nên không rò rỉ tương lai, chỉ có một tham số và phản ứng nhanh hơn trung bình trượt thường khi xu hướng đổi. Span 10 em cố định trước, không chỉnh theo kết quả.

Hỏi: Không có nhãn bất thường thì kiểm chứng thế nào? (slide 12)

Trả lời: Em so mỗi động cơ với 30 cycle đầu của chính nó và đo số cycle báo trước khi hỏng. Em chưa đo được tỷ lệ báo động giả vì mọi động cơ train đều đi đến hỏng. Trên test, mức theo dõi không bỏ sót động cơ nào cần bảo trì, nhưng còn rộng.

Hỏi: Vì sao chọn 3 sigma và 5 cycle liên tiếp? Đổi số khác thì sao? (slide 12)

Trả lời: Hai con số này em cố định trước khi chạy, theo lệ thường của biểu đồ kiểm soát chất lượng: 3 sigma là mức mà nếu chỉ do dao động ngẫu nhiên thì rất hiếm khi vượt, còn 5 cycle liên tiếp để một cái gai lẻ không làm hệ thống la làng. Sau đó em có chạy lại cả lưới 2, 3, 4 sigma nhân với 3, 5, 10 cycle, tổng cộng 9 cấu hình (Bảng 19 trong báo cáo, và có trên dashboard). Kết quả là cấu hình nào cũng bắt đủ 100 trên 100 động cơ, và động cơ báo muộn nhất vẫn còn ít nhất 50 cycle mới hỏng. Cái thay đổi chỉ là đánh đổi. Hạ xuống 2 sigma, 3 cycle thì báo sớm hơn, trung vị 136 cycle, nhưng có 17 động cơ bị báo ngay sau đoạn tham chiếu, giống cái chuông báo cháy kêu khi mới bật bếp. Nâng lên 4 sigma, 10 cycle thì không còn báo quá sớm, nhưng mất khoảng 22 cycle báo trước. 3 sigma, 5 cycle nằm giữa: chỉ 1 trường hợp báo quá sớm, trung vị 121 cycle. Nói gọn: kết luận không phụ thuộc vào con số em chọn.

Hỏi: Vì sao chia GroupKFold theo động cơ? (slide 13)

Trả lời: Hai window liền kề của cùng động cơ trùng 29 trên 30 cycle. Chia ngẫu nhiên theo dòng sẽ đưa các window gần như giống nhau vào cả hai phía, làm sai số đẹp giả tạo.

Hỏi: Có rò rỉ dữ liệu không? (slide 5)

Trả lời: Window, EWMA và quy tắc bất thường chỉ dùng quá khứ; mọi window của một động cơ nằm cùng một fold; Health Index chỉ học trên train; nhãn test chỉ đọc sau khi đã ghi dự đoán và lịch bảo trì.

Hỏi: Vì sao 49% cảnh báo bị trễ? (slide 15)

Trả lời: Động cơ suy giảm nhanh có RUL thật thấp hơn điều mô hình kỳ vọng ở cùng mức sensor, nên dự đoán chạm 30 muộn hơn. Ngưỡng 40 giảm tỷ lệ trễ xuống 6%, đổi lại báo sớm nhiều hơn.

Hỏi: Mức theo dõi tăng cường có quá rộng không? (slide 16)

Trả lời: Có, 41 trên 59 động cơ ở mức này còn trên 60 cycle. Mức này dùng để tăng tần suất kiểm tra, không dùng để lên lịch can thiệp.

Hỏi: Có động cơ nào bị xếp lịch bảo trì sau khi đã hỏng không? (slide 16)

Trả lời: Có, em nói thẳng luôn: 12 trên 100 động cơ có ngày bảo trì dự kiến muộn hơn lúc hỏng thật, từ 4 đến 27 cycle. Nhưng cả 12 chiếc đều còn ít nhất 77 cycle nữa mới hỏng, tức là toàn những động cơ còn xa. Ở vùng đó mô hình hay đoán dư tuổi thọ, giống dự báo thời tiết hai tuần tới thì kém hơn dự báo ngày mai. Trong vận hành, lịch được tính lại mỗi lần có dữ liệu mới, nên những động cơ này sẽ được xếp lại nhiều lần trước khi đến hạn. Bài học em rút ra là không nên chốt lịch xa chỉ từ một lần dự đoán. Còn với 21 động cơ báo cần bảo trì ngay thì đúng cả 21.

Hỏi: Big Data nằm ở đâu khi dữ liệu nhỏ? (slide 6–7)

Trả lời: Ở cách tổ chức: thu thập có kiểm tra mã băm, ETL nhiều file, lưu trữ phân lớp Parquet, SQLite và HDFS, ba cách làm Pandas, PySpark và MapReduce khớp tuyệt đối, và Spark nhanh hơn khoảng 3,4 lần khi dữ liệu tăng 10 lần. Em chưa chạy trên cluster nhiều máy.

Hỏi: Hadoop nằm ở đâu trong bài? Sao chỉ có một node? (slide 7)

Trả lời: Em dựng Hadoop 3.4.3 trong Docker, đúng kiểu Hadoop Sandbox của Chương 2: NameNode, DataNode cho HDFS và ResourceManager, NodeManager cho YARN. File raw được đưa lên HDFS. Job MapReduce viết bằng Python qua Hadoop Streaming tính số cycle, cycle đầu, cycle cuối của từng động cơ; Spark đọc HDFS, ghi tầng Silver dạng Parquet lên HDFS rồi tính lại. Cả hai khớp 200 trên 200 động cơ với Pandas. Một node là vì đề cương ghi không cần dựng cluster thật; nó chứng minh phép tính đúng trên HDFS, chứ không dùng để đo tốc độ. Ví dụ dễ hình dung: map giống chia đống phiếu khám cho từng người ghi "động cơ số mấy, cycle thứ mấy", reduce giống gom phiếu theo từng động cơ rồi lấy cycle lớn nhất là tuổi thọ. Dữ liệu 5,7 MB nằm gọn trong một block 128 MB; nếu lớn hơn, HDFS sẽ chia thành nhiều block và YARN chạy nhiều map song song.

Hỏi: Sao không dùng LSTM hay Transformer? (slide 17)

Trả lời: Trong báo cáo, em đã thử nested CV và thêm đặc trưng lịch sử; không hướng nào cải thiện đồng thời sai số và cảnh báo. Mô hình sâu hơn phải so trên cùng cách đánh giá theo động cơ; em chưa có kết quả đó trong báo cáo nên chưa kết luận.

Hỏi: Kết quả có dùng được cho động cơ thật không?

Trả lời: Chưa. C-MAPSS là dữ liệu mô phỏng, không có chi phí bảo trì hay lịch sử sửa chữa, và mô hình FD001 cũng không chuyển thẳng được sang điều kiện vận hành khác.

Hỏi: Câu không biết trả lời

Trả lời: "Dạ, phần này em chưa kiểm tra trong bài; em xin ghi nhận để bổ sung." Không đoán số liệu.

Số liệu tra nhanh

Nội dung	Số liệu
Dữ liệu FD001	100 động cơ train, 100 test; 20.631 dòng train, 13.096 dòng test; 21 sensor
Pandas vs PySpark	Sai khác 0; ở 10×: Pandas 112 giây, Spark 33 giây (nhanh hơn 3,4 lần)
Hadoop	3.4.3 một node trong Docker; MapReduce trên YARN (3 map, 1 reduce) và Spark đọc HDFS đều khớp 200/200; fsck HEALTHY
Thu thập	Tải từ data.nasa.gov, MD5 gói khớp, 12/12 file đúng SHA-256
Khử nhiễu EWMA	Span 10, $\alpha \approx 0{,}18$; giảm 83,6–86,4% dao động, trung vị 86,3%
Window	30 cycle, 21 × 6 = 126 đặc trưng; 17.731 window train, 10.196 window test
Xu hướng	15/21 sensor thay đổi có ý nghĩa; 12 sensor đổi cùng hướng ở 100/100 động cơ, ví dụ sensor 4, 7, 11, 12, 20
Bất thường	100/100 động cơ train; trung vị 121 cycle trước khi hỏng (72,5 nếu không khử nhiễu); đã qua khoảng 39% vòng đời
Độ nhạy bất thường	9 cấu hình (2/3/4σ × 3/5/10 cycle) đều 100/100; lead time nhỏ nhất $\geq$ 50; 2σ–3 cycle: trung vị 136, 17 báo sớm; 4σ–10 cycle: 98,5, 0 báo sớm
Mô hình	HistGradientBoosting, GroupKFold 5 fold; MAE CV 20,21 (Ridge 23,99)
Kết quả test	MAE 15,35 (CI 95%: 12,73–18,09); RMSE 20,54; NASA 1.513; MAE 4,38 khi RUL thật $\leq$ 30
Cảnh báo ngưỡng 30 (OOF train)	44 trước, 7 đúng lúc, 49 muộn, 0 bỏ sót; báo trước trung vị 30 cycle
Cảnh báo ngưỡng 40	Kịp thời 94% (ngưỡng 30: 51%); trễ 6%
Lịch bảo trì	21 cần bảo trì, 59 theo dõi, 20 bình thường; 21/21 đúng; 14 đến hạn trong 25 cycle tới; 12 động cơ có lịch muộn hơn lúc hỏng 4–27 cycle (RUL thật đều $\geq$ 77)
Ví dụ động cơ 1	Bất thường cycle 84; cảnh báo RUL cycle 162; hỏng cycle 192
Ví dụ động cơ 90	Cycle 146; RUL dự đoán 31,2; bảo trì cycle 147; bất thường từ cycle 67
Chuyển miền	MAE 47,39–60,32 (huấn luyện trên chính subset: 22,36–28,33); tăng 1,7–2,7 lần